

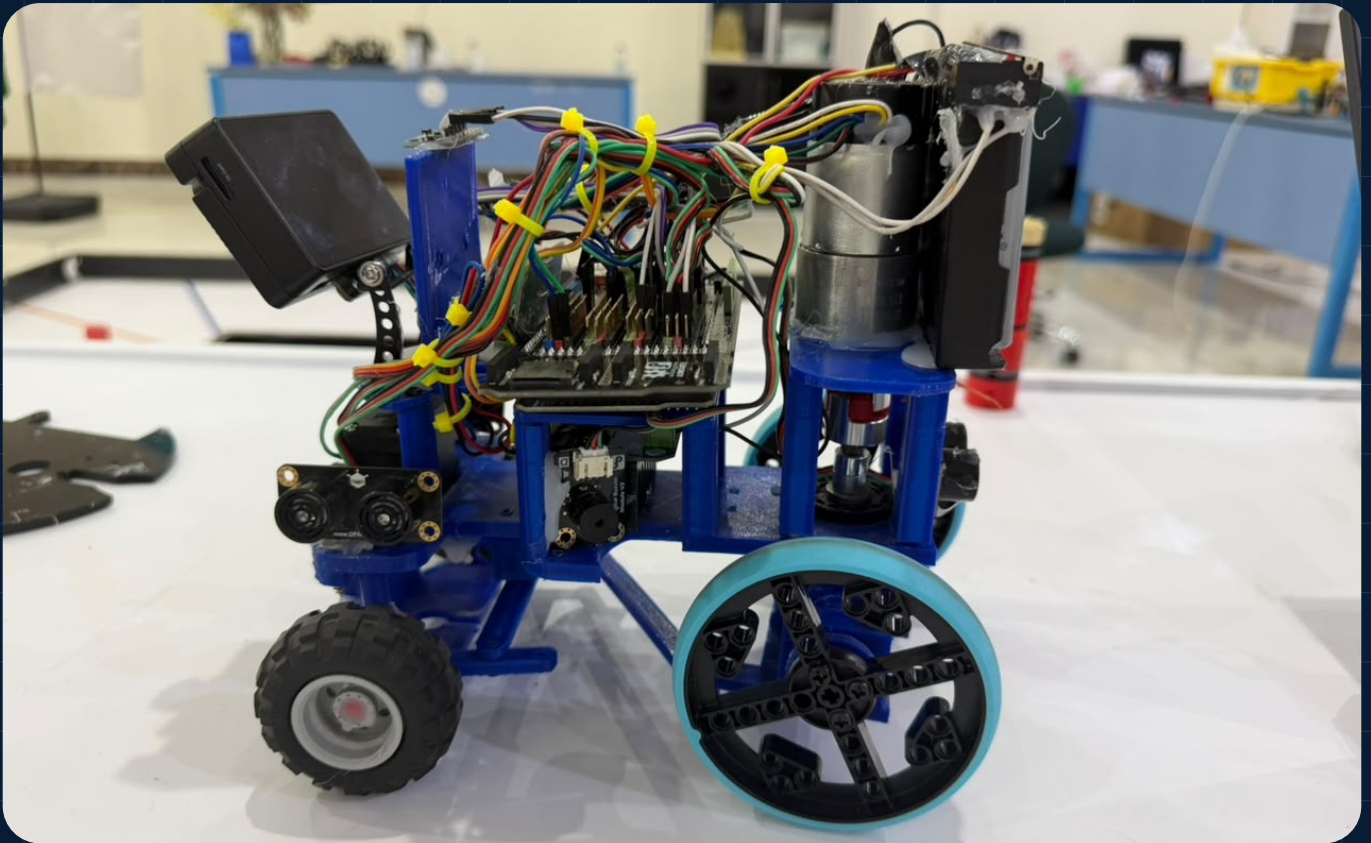
FUTURE ENGINEERS 2026

ROBOT ENGINEERING JOURNAL

As-built mechanical system • electronics • code • tests • field corrections

TEAM MEMBERS

Shaden • Lana



FINAL COMPETITION ENGINEERING DOCUMENTATION

Mechanical build • electronics • autonomous control • testing • validation

BUILD • TEST • MEASURE • IMPROVE

Project scope and engineering approach

The robot was developed for WRO Future Engineers 2026 as an autonomous vehicle integrating a custom mechanical chassis, encoder-based drive control, servo steering, HUSKYLENS vision, MPU6050 yaw feedback, and six URM09 ultrasonic sensors. The system was refined through repeated field testing in both normal navigation and obstacle-challenge conditions.

Engineering decisions were verified using physical measurements and Serial telemetry. Mechanical behavior, sensor readings, state transitions, corner accuracy, pillar avoidance, and final parking distances were tested independently so each correction could be evaluated on the real robot.

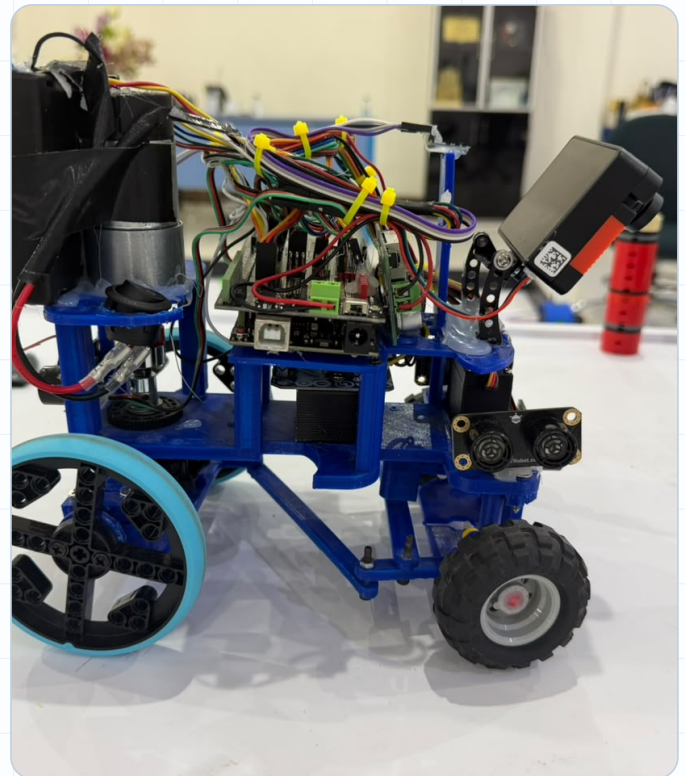
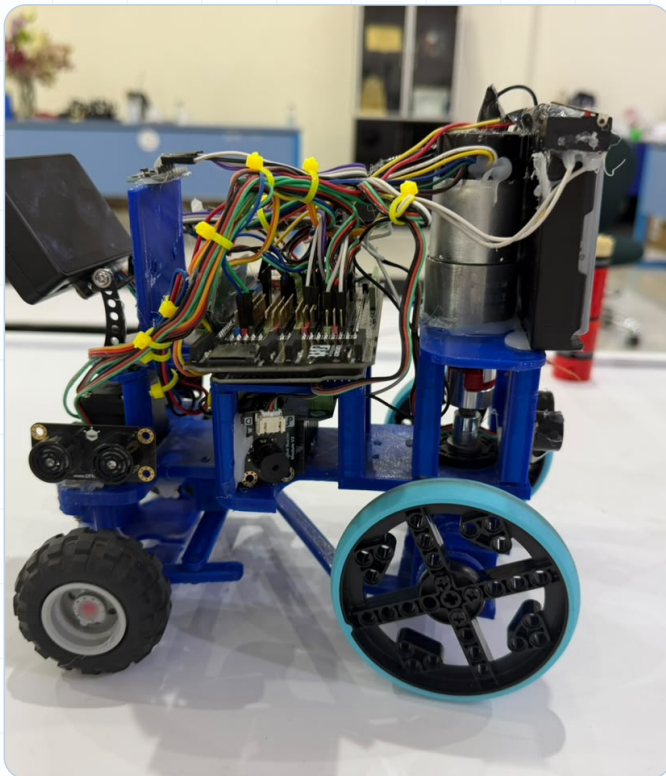
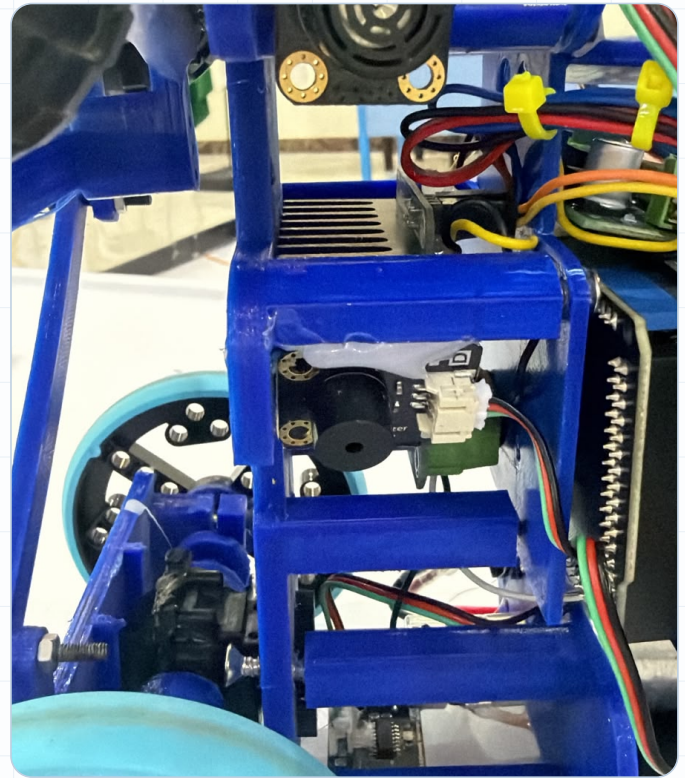
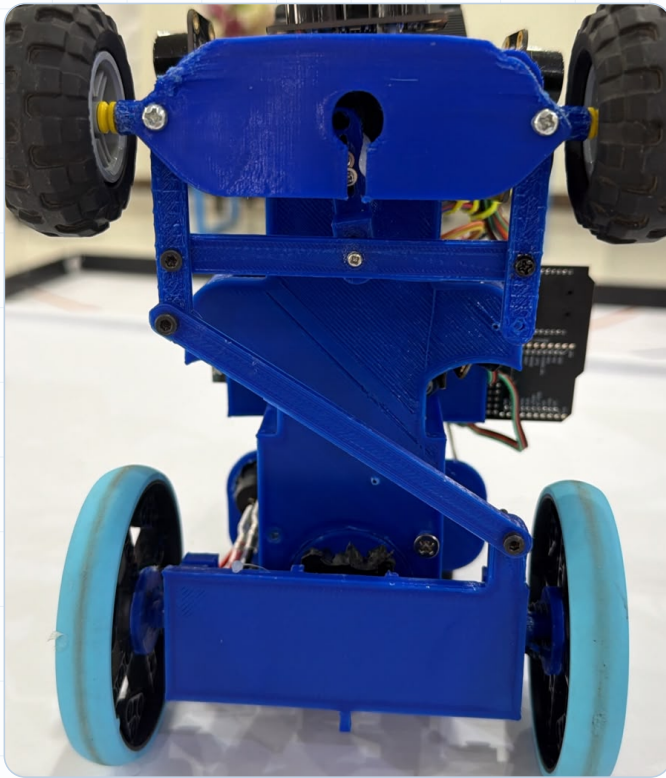
Team members

Shaden and Lana worked together on the mechanical build, electronics integration, programming, field testing, tuning, troubleshooting, and technical documentation of the robot.

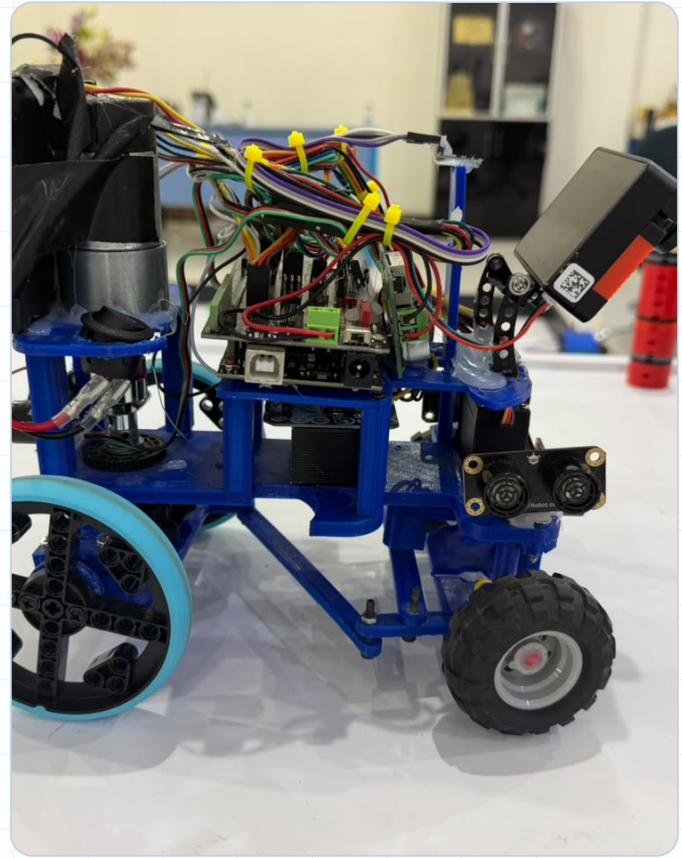
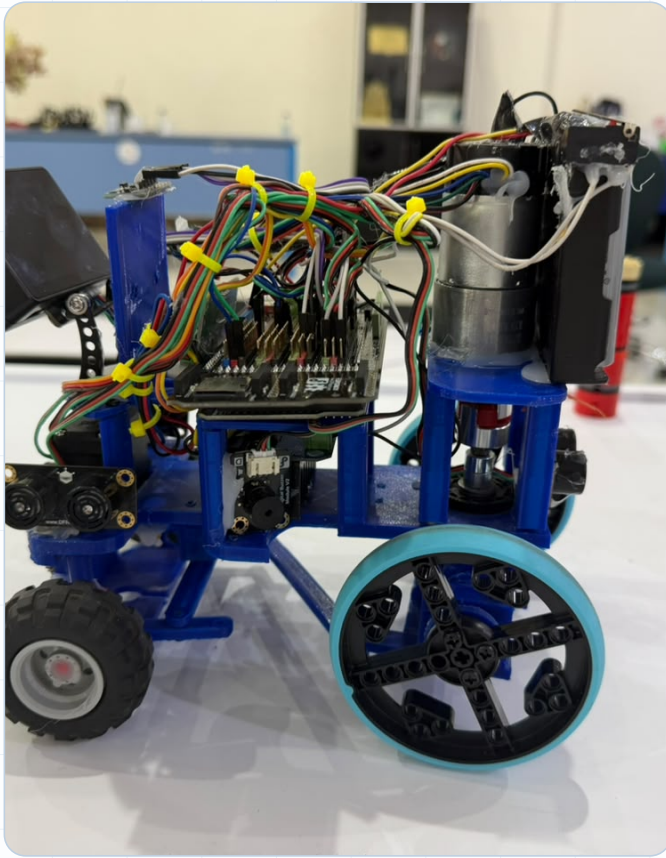
Working method

- Change one main variable at a time during tuning.
- Use Serial telemetry to check what the program thinks is happening.
- Measure the physical result on the field rather than assuming a software target equals a real angle or distance.
- Keep a known working version before making a new change.
- Separate normal driving logic from obstacle and final-parking logic so one feature does not accidentally break another.

4	Current robot: photographic record	19	Pillar avoidance
5	Mechanical architecture	20	Final parking sequence
6	Front steering mechanism	21	Current tuning sheet
7	Drive motor and encoder	22	Field test procedure
8	Electronics packaging	23	Issue log: steering and yaw
9	Power architecture	24	Issue log: intersections and pillars
10	Arduino Mega pin map	25	Issue log: reverse and final distance
11	Sensor layout	26	Student work: hardware
12	HUSKYLENS vision logic	27	Student work: programming and testing
13	MPU6050 yaw control	28	Development timeline
14	Ultrasonic strategy	29	Pre-competition checklist
15	Software architecture	30	Known limitations / next work
16	State machine	31	Code baseline and revision note
17	Intersection and direction logic	32	Engineering summary
18	Corner-turn control	33	Team sign-off

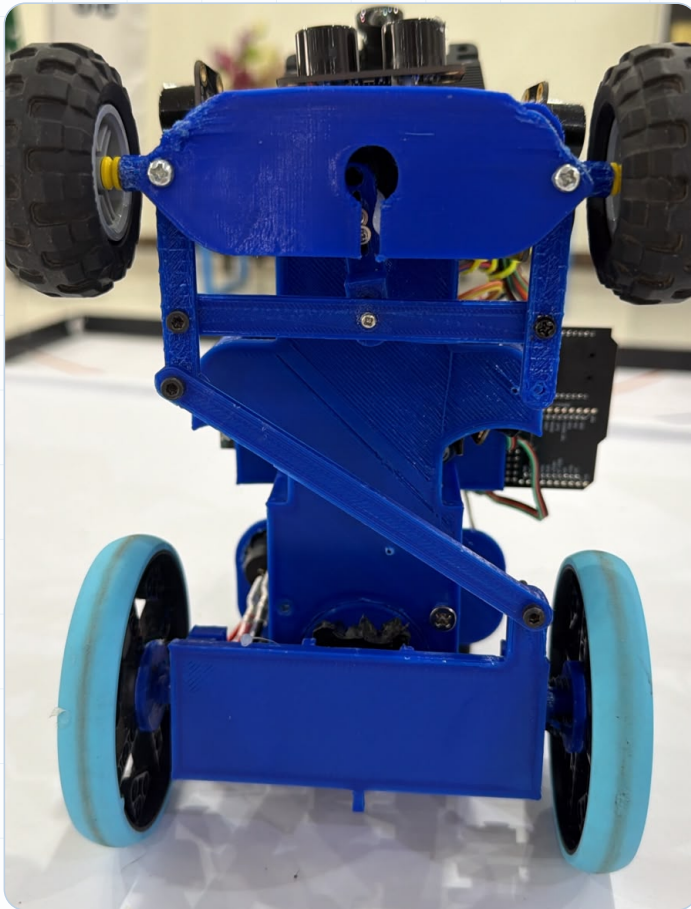


Four reference views of the current robot. These images are treated as the primary mechanical evidence in this report.



What the photographs show

- **Custom blue chassis:** layered printed structure with vertical posts and cross-bracing.
- **Front steering axle:** two small front wheels mechanically linked to one servo-driven steering system.
- **Rear traction section:** larger wheels and a geared motor/encoder assembly mounted toward the rear.
- **Electronics deck:** controller board and interconnects placed high and central for access during debugging.
- **Vision and distance sensors:** camera and ultrasonic modules mounted at the edges where they have a clear field of view.
- **Battery and motor packaging:** high-current components grouped near the drive section to reduce long power paths.

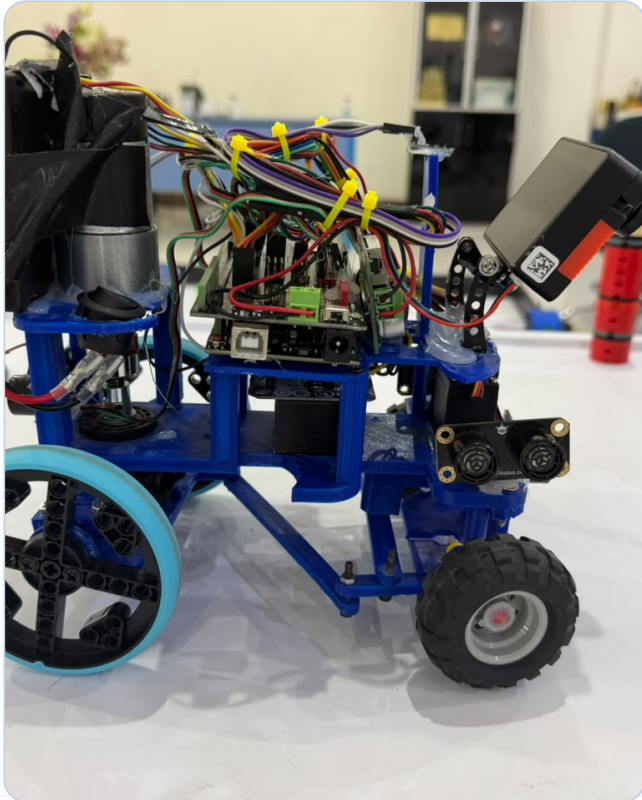


How steering is controlled

- Software center reference: 88° .
- The steering command is converted to a servo angle around the center reference.
- The front wheels are linked mechanically, so one servo command changes both front wheel angles together.
- Mechanical play, tire friction, servo speed, and chassis weight all affect the final physical heading.
- The code reduces steering as the yaw target is approached to reduce overshoot.

Why this mattered in testing

A software target of 90° does not guarantee a physical 90° turn. We therefore separated **yaw measurement calibration** from **turn-shape tuning**. This made it possible to keep the target heading meaningful while changing how aggressively the servo approaches it.



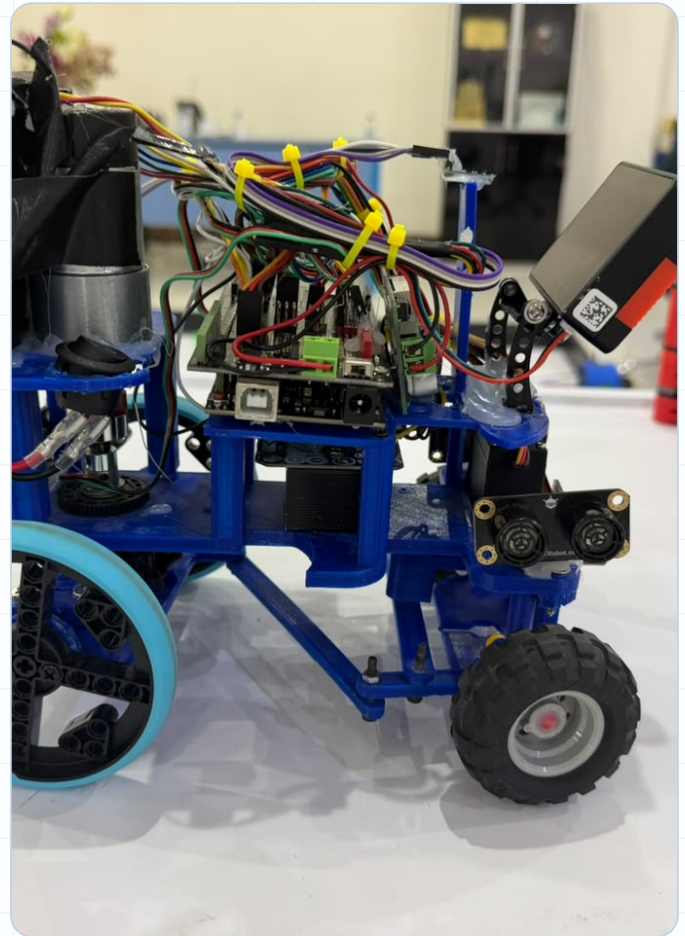
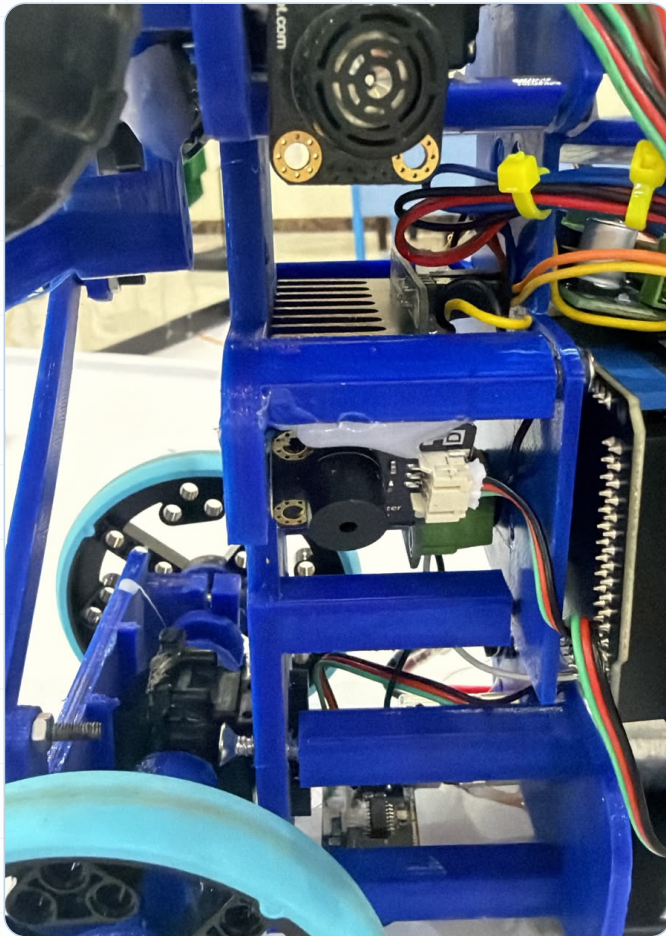
Drive subsystem

- DFRobot FIT0186 / GB37Y3530-12V-251R geared DC motor.
- BTS7960 high-current motor driver.
- Hall encoder connected to D2 / D3.
- Current distance calibration uses 700 counts per wheel revolution and 7.0 cm wheel diameter.
- Encoder travel is used for repeatable approach, reverse, and positioning movements.

Distance model

```
countsPerCM = CPR / ( $\pi \times \text{wheelDiameter}$ )  
distanceCM = |currentTicks - startTicks| / countsPerCM
```

Recalibration rule:
 $\text{newCPR} = \text{oldCPR} \times \text{commandedDistance} / \text{actualDistance}$



Design choices visible in the final build

- Controller and interconnect boards are accessible from the side for quick cable checks.
- Wire bundles are tied together instead of left loose near the wheels and steering links.
- The vision sensor is mounted away from the main wire bundle so its view is not blocked.
- High-current drive components are kept physically close to the motor and battery.
- Sensor connectors remain visible enough to replace a module without dismantling the complete chassis.



Power rules used on the robot

- Motor supply and logic supply are separated so the Arduino and sensors are not powered directly from the 12 V motor line.
- All grounds share a common reference.
- Motor-current wiring is kept short where practical.
- The 5 V rail supplies the controller, camera, IMU, ultrasonic sensors, buzzer, and servo electronics.

Device / signal	Pin	Interface
Encoder A	D2	Interrupt
Encoder B	D3	Digital
BTS7960 RPWM	D5	PWM
BTS7960 LPWM	D7	PWM
Steering servo	D9	Servo PWM
HUSKYLENS TX -> Mega RX1	D19	UART
HUSKYLENS RX -> Mega TX1	D18	UART
MPU6050 SDA	D20	I2C
MPU6050 SCL	D21	I2C
Front URM09	D22	Single-wire
Left URM09	D23	Single-wire
Right URM09	D24	Single-wire
Rear URM09	D25	Single-wire
Start button	D26	Active LOW
Rear-left URM09	D27	Single-wire
Rear-right URM09	D28	Single-wire
Buzzer	D29	Digital output

HUSKYLENS 2

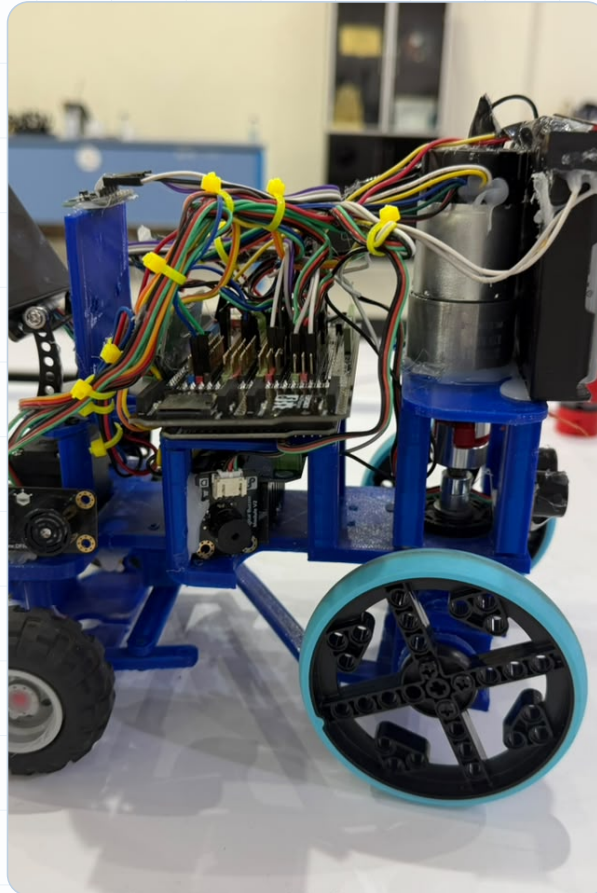
Intersection and pillar recognition

Front URM09

Corner and final distances

Rear-side URM09

Pillar pass and rear clearance



MPU6050

Yaw / heading estimate

Side URM09

Opening / wall context

Encoder

Travel distance

Sensor fusion rule

The robot does not ask one sensor to solve every problem. Vision identifies what the robot is seeing, the IMU tracks heading, ultrasonics measure surrounding distance, and the encoder measures travel. The control program combines those roles inside the state machine.

400

MINIMUM LINE AREA

80

MINIMUM LINE Y

155.0

PILLAR CLOSENESS

60 ms

POLL INTERVAL

Current logic

- The first accepted intersection decides the race direction: BLUE -> LEFT, ORANGE -> RIGHT.
- After the first completed turn, the direction stays latched. Later BLUE or ORANGE markers can both mean "intersection", but the robot still turns in the learned direction.
- The intersection filter rejects small/remote detections using line area and vertical image position.
- Red and green learned IDs are used for obstacle pillars. Pillar avoidance uses camera geometry rather than front-ultrasonic distance as its main proximity decision.
- Once an intersection is accepted, pillar avoidance is blocked until the corner turn finishes.

88°

SERVO CENTER

2.60

CORRIDOR YAW_KP

90.0°

CORNER TARGET

2.0°

CORNER TOLERANCE

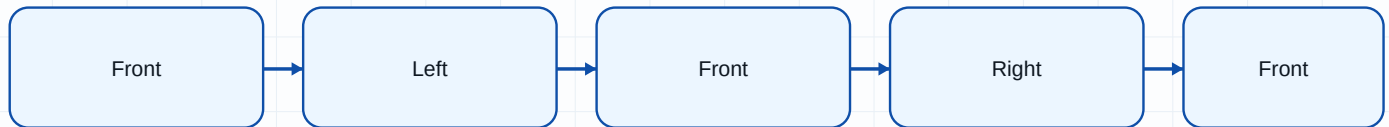
What was calibrated

We first verified the sign convention: rotating the robot left must increase yaw. After that, yaw scale and turn behavior were treated as separate problems. The program keeps a base YAW_SCALE for normal heading estimation and a turn-specific multiplier during the actual corner rotation.

Current turn-only multiplier: **1.78**
Base yaw scale: **1.0000**
Gyro deadband: **0.08 deg/s**

Practical interpretation

If the Serial yaw says 90° and the chassis is physically not at 90°, the problem is not solved by blindly changing the target angle. We compare measured physical rotation, software yaw, steering speed, and mechanical overshoot before choosing the parameter to adjust.



Why the scheduler matters

A URM09 measurement uses a blocking pulse-in operation. Reading all six sensors every loop would slow the control system. The program therefore reads one sensor per slot and gives the front sensor extra slots during corner and final-distance states.

Front-distance confirmation

- Corner front verification uses fresh scheduler samples instead of creating extra sonar reads.
- The final center distance uses a short 3-sample window with 2 required hits.
- Invalid/no-echo readings do not count as a real distance.
- After the final 90° corridor turn, the robot first moves 1 second with yaw correction before activating the 260.0 cm front-distance check.
- After the parking-entry turn, another 1-second yaw-correction period occurs before the final 10.0 cm stop check.

TUNING PANEL

Angles, distances, gains, thresholds, timeouts

STATE MACHINE

Corridor, approach, turning, pillar, final parking

DECISION LAYER

Intersection priority, pillar priority, fallback rules

SENSOR PROCESSING

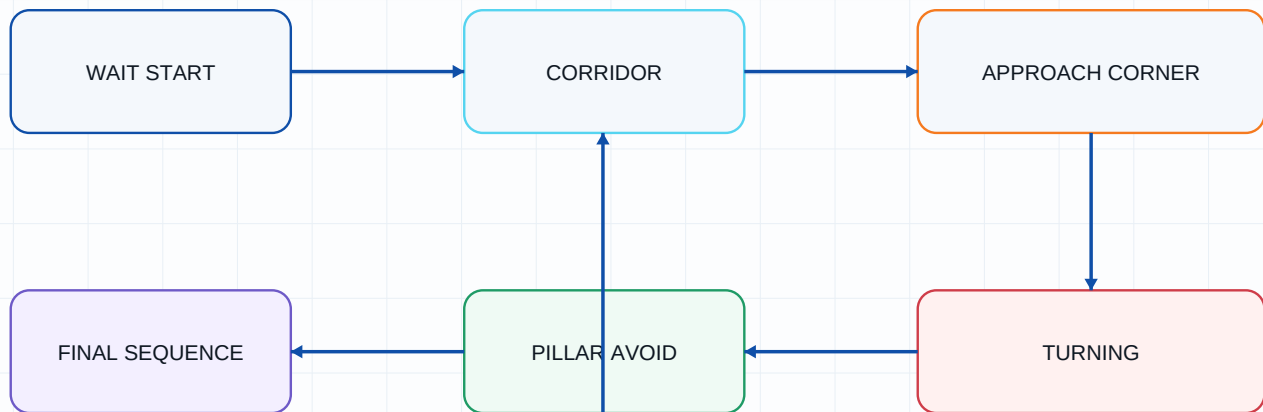
Yaw, filtered distance, HUSKYLENS results, encoder travel

HARDWARE I/O

PWM, servo, UART, I2C, ultrasonic timing, button, buzzer

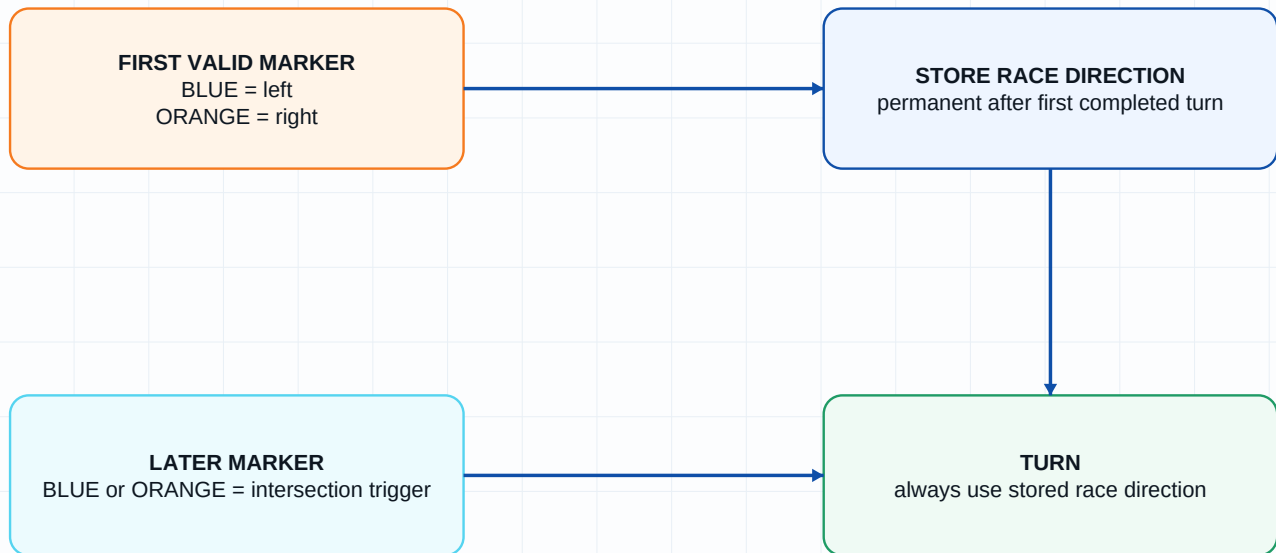
Reason for the structure

The state machine keeps critical behaviors from competing with each other. A confirmed corner has a clear sequence. Pillar avoidance has its own sequence. Final parking has its own sequence. This makes testing easier because a change can be limited to one behavior instead of changing the whole program.



Main transition rules

- Button press leaves WAIT START and enters corridor driving.
- A valid intersection moves the robot into corner approach.
- Once an intersection is accepted, pillar avoidance is temporarily blocked.
- A close pillar can start pillar avoidance only when no accepted corner is already being completed.
- After the required turn count, obstacle mode enters the special final parking sequence.



Why we changed this

During testing, the camera could sometimes recognize the opposite line color at a later corner. After the first corner, the robot no longer needs the line color to choose left or right. It only needs proof that it has reached an intersection. The stored race direction provides the turn direction.

Corner priority over pillars

The live intersection check is evaluated before pillar avoidance. During APPROACH_CORNER and TURNING, pillar avoidance is intentionally ignored until the corner is finished.

90.0°

CORNER TARGET

2.0°

TOLERANCE

1.78

TURN MULTIPLIER

140 ms

BRAKE SETTLE

Current corner trigger values

After the camera sees an intersection, the robot keeps the old corridor yaw until the active trigger becomes true. During the actual turn, the steering controller starts stronger and softens as the remaining yaw decreases. A short brake/settle phase helps the steering return toward center before corridor correction takes over.

Encoder forward distances

[0, 75, 75, 0] cm

Front-ultrasonic thresholds

[220, 35, 35, 220] cm

Side opening gate

TURN_SIDE_OPEN_MIN_CM = 5

155.0

CAMERA CLOSENESS

40.0°

MAX AVOID YAW

40.0°

TURN STEERING

2.80

PILLAR YAW_KP

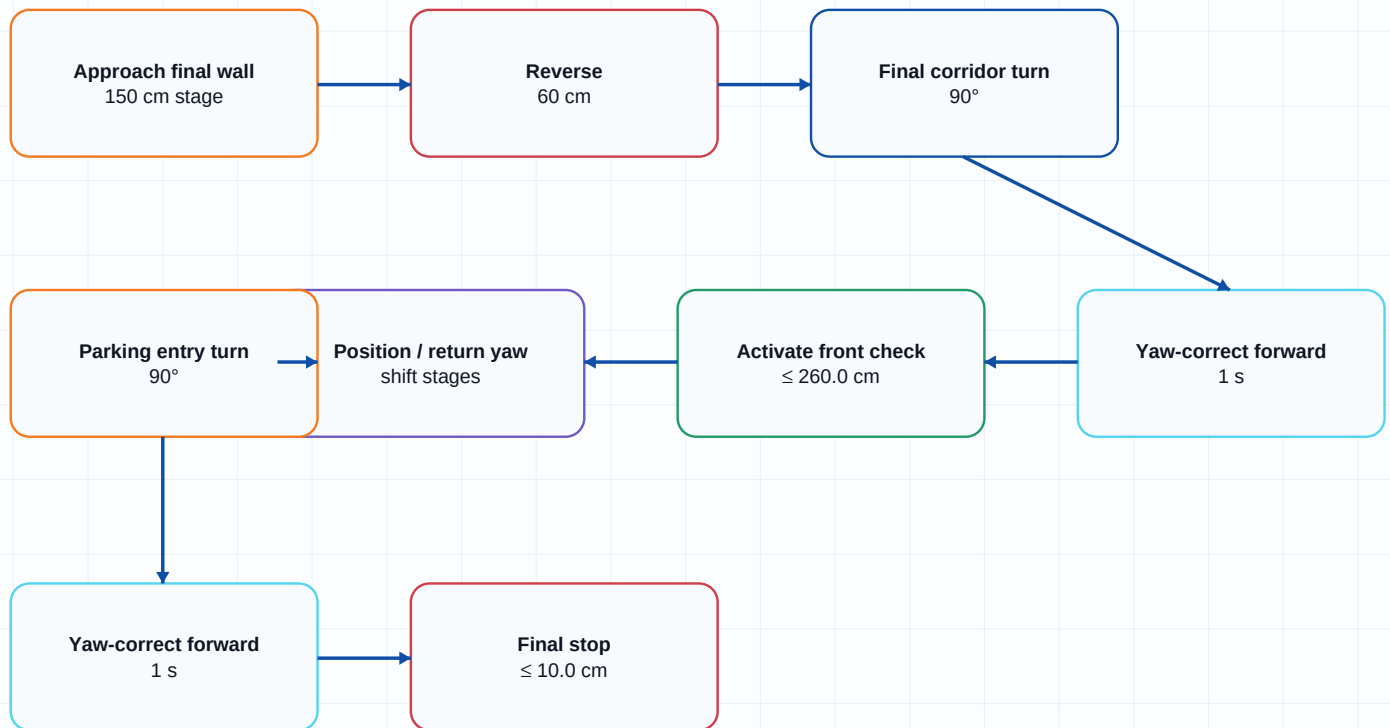
150

PASS PWM

- 1 Detect a valid red/green pillar close enough to matter.
- 2 Stop briefly and pre-steer.
- 3 Turn out toward an avoidance yaw target.
- 4 Drive beside the pillar while holding yaw.
- 5 Use side/rear-side evidence to confirm the pillar has passed.
- 6 Return to the main corridor yaw.

Safety rule used in the current code

If a corner has already been accepted, pillar avoidance is not allowed to interrupt it. This was added because a visible pillar near a corner could previously cause the robot to abandon the turn sequence.



Why the 260.0 cm stage was changed

The previous version required the front sensor to first see a value greater than 265 cm before 260.0 cm could become active. On the real robot that condition was not always reached. The current sequence instead uses a 1-second yaw-correction period, resets the verifier, and then evaluates fresh front samples directly against 260.0 cm.

Parameter	Current value	Purpose
SERVO_CENTER_DEG	88°	Straight steering reference
YAW_KP	2.60	Corridor yaw correction
TURN_ANGLE_DEG	90.0°	Nominal corner target
TURN_TOLERANCE_DEG	2.0°	Turn completion tolerance
FOUR_WS_TURN_YAW_MULTIPLIER	1.78	Turn-only yaw calibration
HUSKY_LINE_MIN_AREA	400	Minimum accepted intersection area
HUSKY_LINE_MIN_Y	80	Minimum intersection vertical position
PILLAR_CAMERA_CLOSENESS	155.0	Pillar visual closeness
PILLAR_TUNE_MAX_TURN_DEG	40.0°	Maximum avoidance yaw
PILLAR_TURN_STEER_DEG	40.0°	Steering during pillar turn
PILLAR_YAW_KP	2.80	Yaw hold during pillar maneuver
FINAL_CENTER_FRONT_CM	260.0 cm	Final center front threshold
FINAL_CENTER_YAW_CORRECT_BEFORE_FRONT_MS	1000 ms	Yaw correction before final-center check
FINAL_PARK_STOP_FRONT_CM	10.0 cm	Stop inside parking
FINAL_PARK_YAW_CORRECT_BEFORE_FRONT_MS	1000 ms	Yaw correction before final stop check

1. Make one change

Change a single parameter or one logic block. Save the previous working file.

2. Reproduce the same test

Use the same start position and field setup so the result can be compared.

3. Watch telemetry

Check state, yaw, ultrasonic, encoder, and vision values while the robot runs.

4. Measure the physical result

Measure angle, distance, or stopping position on the field.

5. Record the effect

Write whether the change improved, worsened, or did not affect the problem.

6. Keep or revert

Keep the change only if the result is repeatable. Otherwise restore the previous working setting.

OBSERVATION After some turns the robot drifted and the physical turn did not match the expected 90° heading.

CHANGE We verified yaw sign, reduced the yaw scale to a sensible baseline, used a turn-only multiplier, and separated corridor correction from corner-turn calibration.

RESULT / CHECK Serial yaw, physical heading, and servo response can now be tuned independently instead of compensating one problem with another.

OBSERVATION The corridor correction sometimes felt too slow or too aggressive.

CHANGE We exposed YAW_KP, steering limits, deadbands, and post-turn correction parameters so response could be adjusted without changing the state machine.

RESULT / CHECK The robot can be tuned for faster recentering while keeping the corner logic unchanged.

OBSERVATION The robot sometimes continued moving while the steering mechanism was still returning to center after a turn.

CHANGE A short turn brake/settle period was kept in the corner controller.

RESULT / CHECK The next corridor starts with a more stable steering position.

OBSERVATION A 90° software target did not always equal a 90° physical chassis rotation.

CHANGE We treated gyro calibration and mechanical overshoot as separate causes and compared the serial “rotated” value to the physical result.

RESULT / CHECK This made the tuning process measurable instead of changing TURN_ANGLE_DEG blindly.

- OBSERVATION** A close pillar could interrupt an already accepted corner.
- CHANGE** The live intersection check was moved ahead of pillar avoidance, and pillar interruption was removed from APPROACH_CORNER.
- RESULT / CHECK** Once the robot commits to a corner, it completes the corner before avoiding another pillar.

- OBSERVATION** Later intersections could be seen as the opposite line color even though the race direction was already known.
- CHANGE** After the first completed turn, either blue or orange is accepted as an intersection trigger; the stored race direction decides left or right.
- RESULT / CHECK** The robot no longer depends on later color recognition to remember the round direction.

- OBSERVATION** Pillar avoidance could start too early or too late.
- CHANGE** The visual closeness threshold was simplified to a camera-height based tune and the steering/yaw values were exposed as separate parameters.
- RESULT / CHECK** Detection distance and avoidance strength can now be adjusted independently.

- OBSERVATION** A saved intersection after a pillar could cause an incorrect extra travel distance.
- CHANGE** The code remembers the encoder tick where the intersection was first seen and carries the already-traveled distance into the resumed corner approach.
- RESULT / CHECK** The robot does not repeat the full approach distance after finishing the pillar maneuver.

OBSERVATION The robot suddenly stopped and drove backward during normal corridor driving.

CHANGE The automatic stuck/escape reverse state machine was removed completely from the working baseline.

RESULT / CHECK Normal driving no longer enters an unexpected reverse because of the old stuck detector.

OBSERVATION The final center value of 260.0 cm did not activate reliably.

CHANGE The “must first see >265 cm” arming rule was removed. The robot now yaw-corrects for 1 second, resets the front verifier, and then checks fresh samples.

RESULT / CHECK The 260.0 cm value now acts as the actual threshold instead of depending on a separate larger reading.

OBSERVATION The final parking stop could react before the chassis had finished aligning after the parking-entry turn.

CHANGE The robot now moves forward for 1 second with yaw correction before the final front-distance check is armed.

RESULT / CHECK The 10.0 cm stop is evaluated after the robot has had time to stabilize its heading.

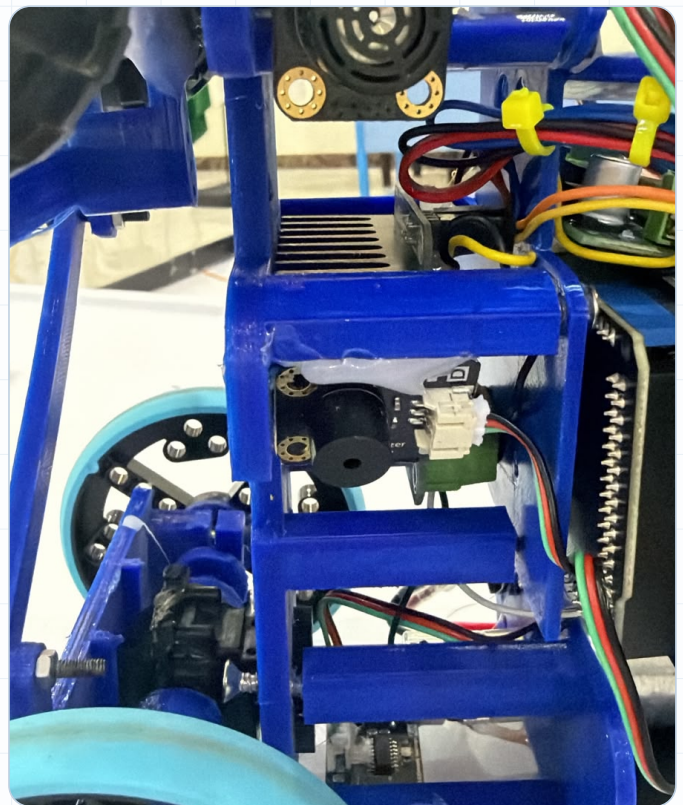
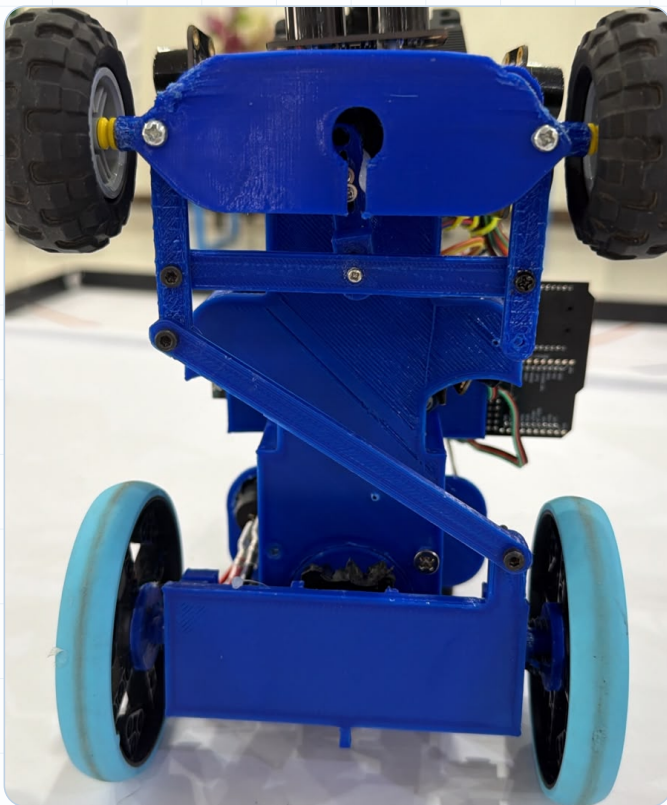
OBSERVATION Noisy front readings could change state too quickly.

CHANGE Critical front distances use short confirmation windows based on fresh scheduler samples.

RESULT / CHECK One isolated low reading is less likely to move the robot into the next final-parking state.

Joint team work recorded from the project

- Built and refined the custom blue chassis and mechanical supports.
- Mounted the front steering system, rear drive assembly, motor, battery, and electronics.
- Installed the HUSKYLENS, MPU6050, six ultrasonic sensors, buzzer, start button, and encoder wiring.
- Secured and reorganized wiring so it remained clear of moving parts.
- Adjusted sensor placement based on what the camera and ultrasonic modules could actually see on the field.
- Checked steering movement, wheel clearance, and mechanical response after each software tuning session.
- Prepared the robot for repeated floor tests and documented the final build with photographs.



Joint programming and field-test work

- Verified motor direction, encoder sign, steering center, start button, buzzer, IMU, camera, and ultrasonic sensors.
- Observed Serial telemetry while moving the robot by hand and while running on the field.
- Tuned corner angle behavior, corridor correction, front-distance triggers, and turn approach distances.
- Adjusted HUSKYLENS line-area and line-position thresholds to reduce false intersection detections.
- Tested red/green pillar behavior and tuned camera closeness, avoidance yaw, steering strength, and pass speed.
- Detected and removed the unwanted automatic reverse behavior.
- Changed corner priority so a confirmed intersection is finished before pillar avoidance.
- Changed later marker logic so either color can represent an intersection after race direction is learned.
- Reworked the final 260 cm and final 10 cm ultrasonic sequences using timed yaw correction and fresh sample verification.
- Compared each code revision against the previous working version before continuing.

What this shows about the team process

The work was iterative. The important skill was not writing one long program once; it was diagnosing what the robot actually did, locating the part of the code responsible, changing the smallest necessary section, and testing again. That pattern is visible in the issue logs on the previous pages.



Before each full run

- Battery charged and securely mounted.
- Steering wheels physically centered at servo center command.
- Encoder and motor direction checked.
- HUSKYLENS lens clean and learned IDs visible.
- Ultrasonic sensors aligned and not blocked by wiring.
- MPU calibration completed while robot is still.
- Correct challenge mode selected.
- Serial telemetry checked for reasonable yaw and distance values.

Known items to keep monitoring

- Physical 90° accuracy can change with tire grip and battery condition.
- Long-range ultrasonic readings near 260 cm may vary with wall angle and surface.
- Camera detection depends on lighting and marker orientation.
- Loose steering linkage can make a correct software command look wrong mechanically.
- Any major mechanical change should be followed by a fresh yaw/turn validation.

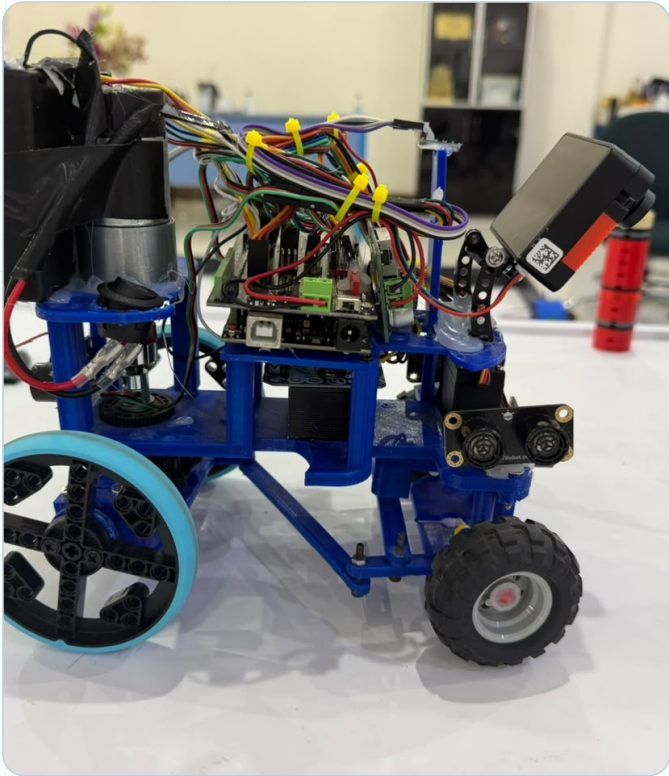
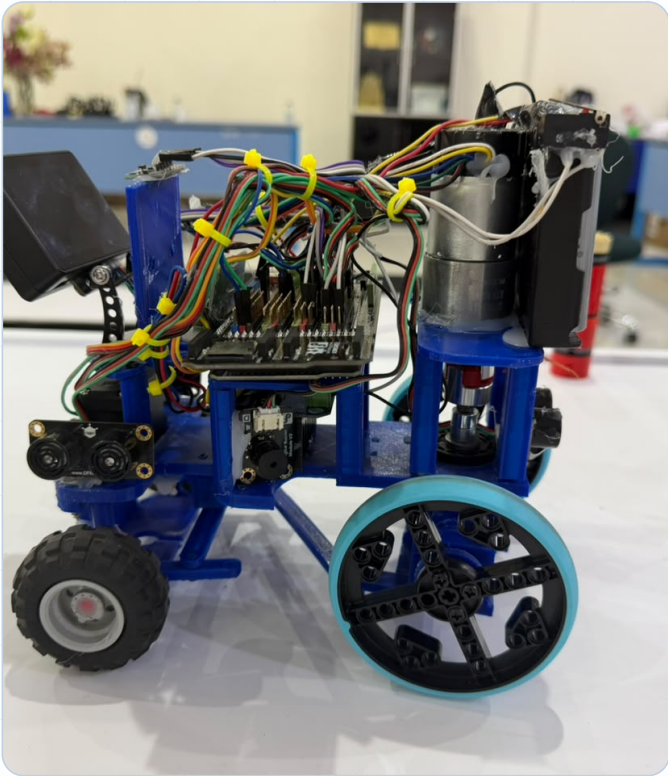
Decision before the competition

Freeze a known working version once the robot completes repeatable runs. Only make a late change if the problem can be reproduced clearly and the new result is verified more than once. The objective is repeatability, not the largest number of last-minute features.

Current software baseline

WRO 2026_FINAL_CENTER_260_FIXED_1SEC_THEN_FRONT_FULL.ino

This report uses the settings and logic from the current working baseline at the time of preparation. The important final behaviors documented here are the learned race direction, corner priority over pillar avoidance, no automatic stuck/reverse escape, the 1-second final-center yaw correction before the 260.0 cm check, and the 1-second parking-entry yaw correction before the 10.0 cm stop.



Project conclusion

The final robot combines a custom mechanical platform, reliable electrical integration, vision-based intersection and pillar recognition, yaw-controlled navigation, encoder feedback, and verified ultrasonic distance stages. The completed system reflects the team's engineering work from assembly and calibration through autonomous testing and final competition preparation.

Team members:

Shaden _____ Lana _____

Date: _____ Coach / reviewer: _____